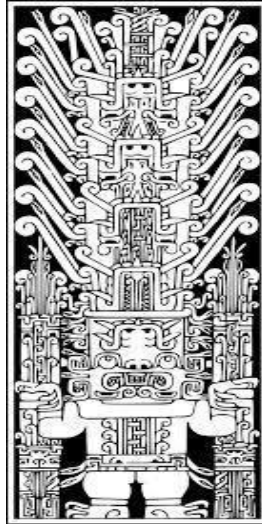


**UNIVERSIDAD NACIONAL FEDERICO VILLARREAL
FACULTAD DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS**



INFORME DE DEFENSA EN PROFUNDIDAD

AUTOR

Quispe Mescua George Angel

CURSO

Ciberseguridad

2026

1. Introducción

El presente informe documenta el desarrollo de un laboratorio práctico orientado a evidenciar la aplicación del modelo de seguridad Defense in Depth (Defensa en Profundidad) sobre una aplicación web intencionalmente vulnerable, OWASP Juice Shop. El objetivo es demostrar, de forma práctica y medible, cómo la incorporación progresiva de capas de seguridad reduce la superficie de ataque y contiene amenazas que en un escenario sin controles se ejecutan sin resistencia.

Adicionalmente, se incluye un análisis comparativo entre el modelo tradicional de Defensa en Profundidad y el modelo emergente de Cybersecurity Mesh Architecture (CSMA), con el fin de identificar fortalezas, limitaciones y escenarios de aplicación de cada enfoque.

2. Objetivos

- Desplegar OWASP Juice Shop en un entorno local controlado mediante contenedores Docker.
- Ejecutar ataques representativos (inyección SQL, Cross-Site Scripting y fuerza bruta) sobre la aplicación sin ningún control de seguridad implementado.
- Implementar capas sucesivas de defensa (segmentación de red, proxy inverso, firewall perimetral y limitación de tasa de peticiones) siguiendo el modelo de Defensa en Profundidad.
- Repetir los ataques iniciales sobre el entorno protegido para verificar la efectividad de cada capa.
- Comparar los resultados obtenidos con los principios de la arquitectura de Cybersecurity Mesh.

3. Marco Teórico

3.1 Defensa en Profundidad

Modelo de seguridad centralizado y por capas concéntricas, en el que cada capa (perímetro, red, host, aplicación y datos) actúa como una barrera adicional frente a un atacante. Su premisa es que ningún control individual es infalible, por lo que la combinación de varios reduce la probabilidad de un compromiso total. Este modelo asume la existencia de un perímetro de red claramente definido.

3.2 Cybersecurity Mesh Architecture (CSMA)

Modelo de seguridad distribuido y centrado en la identidad, en el que cada activo, usuario o servicio cuenta con su propio perímetro de seguridad, independientemente de su ubicación física o de red. Los puntos de control se integran mediante analítica de seguridad compartida, gestión de identidad distribuida y políticas centralizadas con aplicación descentralizada. Surge como respuesta a la disolución del perímetro tradicional en entornos híbridos, distribuidos y con trabajo remoto.

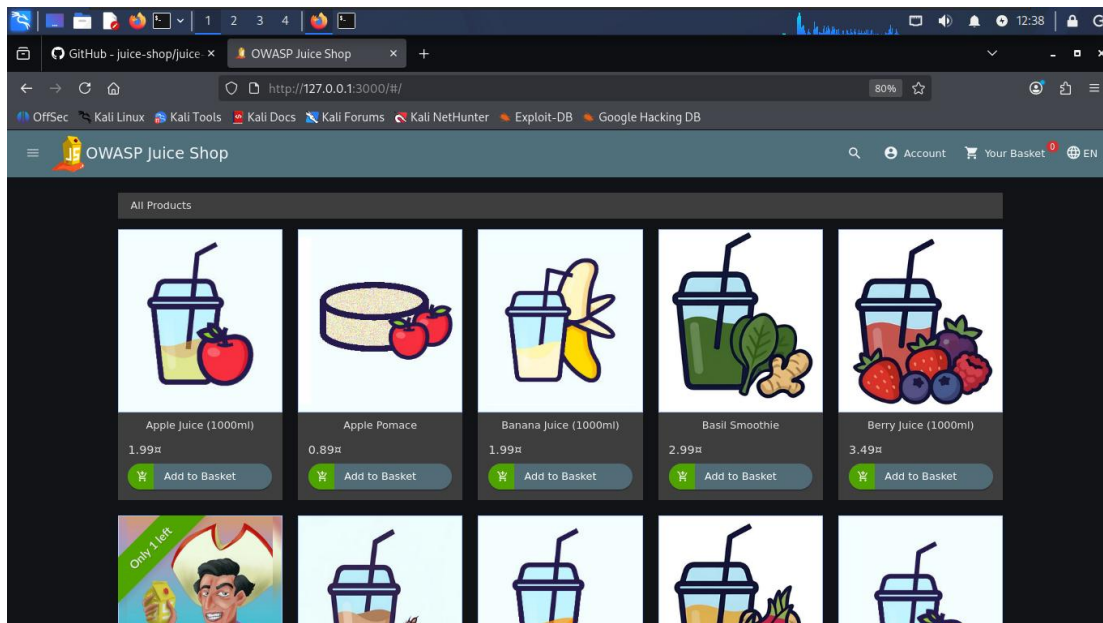
4. Entorno del Laboratorio

- Aplicación objetivo: OWASP Juice Shop (imagen oficial bkimminich/juice-shop), ejecutada en contenedor Docker.
- Equipo atacante: Kali Linux, con Burp Suite Community Edition como herramienta principal de explotación e intrusión.
- Componentes de defensa: red Docker dedicada (juice_net), proxy inverso Nginx, firewall de host (ufw) y Web Application Firewall basado en ModSecurity con OWASP Core Rule Set.

5. Fase 1: Evaluación sin Controles de Seguridad

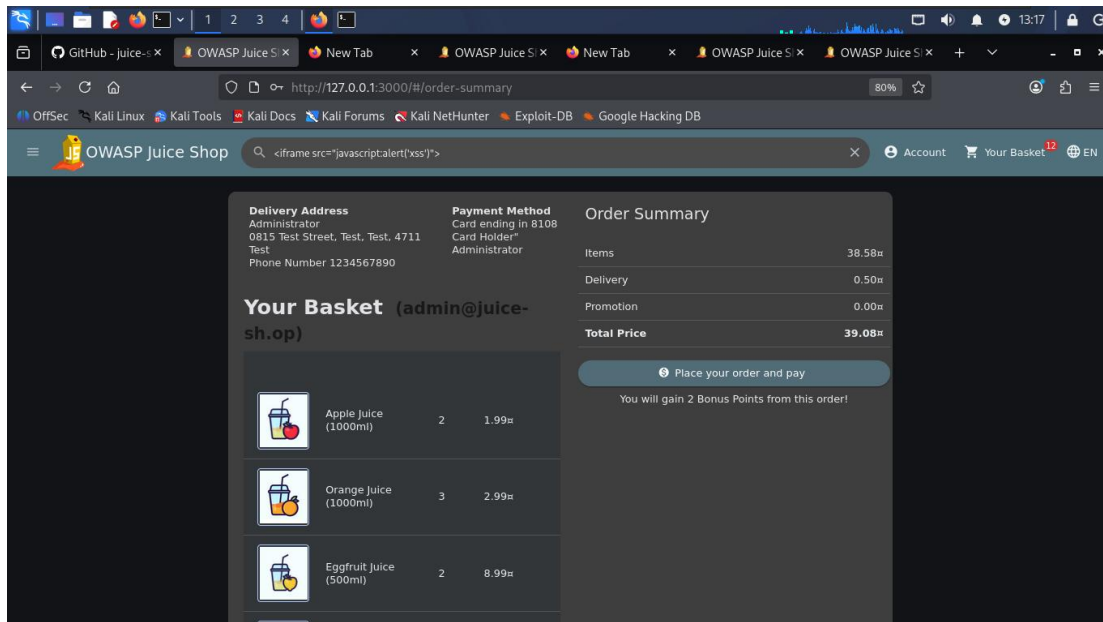
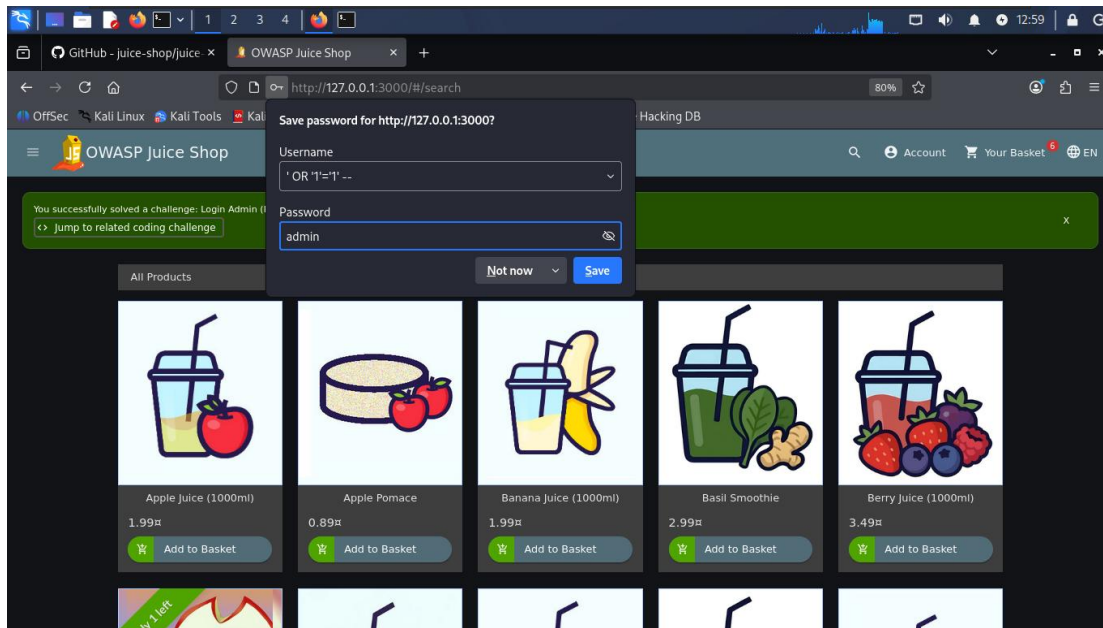
En esta fase se accedió directamente a la aplicación Juice Shop expuesta sin ningún mecanismo de protección, con el objetivo de establecer una línea base de vulnerabilidad.

5.1 Acceso inicial a la aplicación



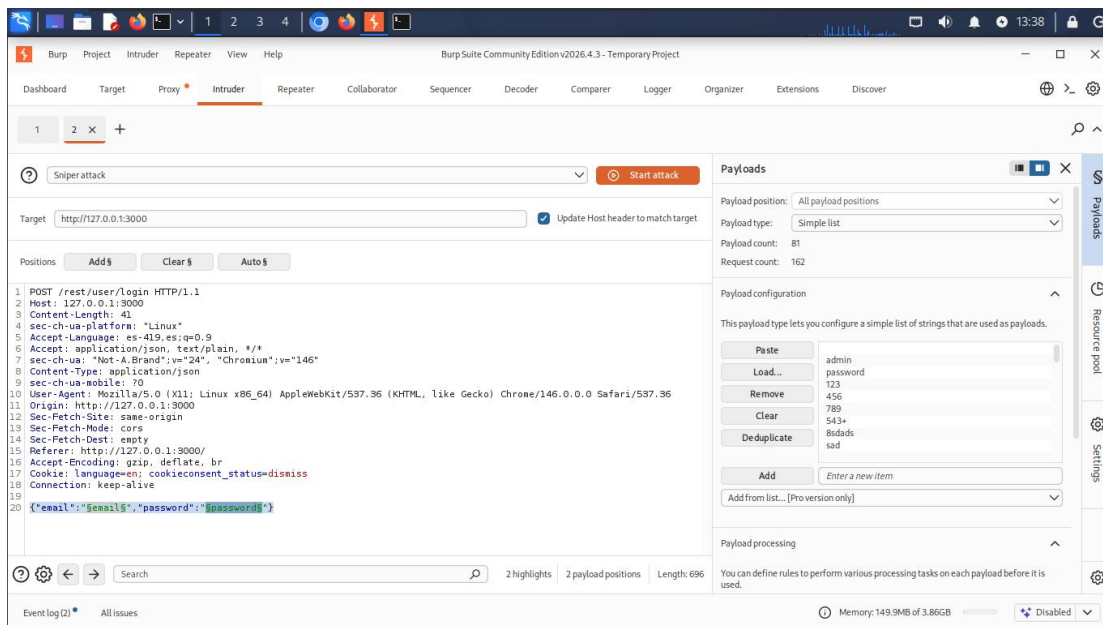
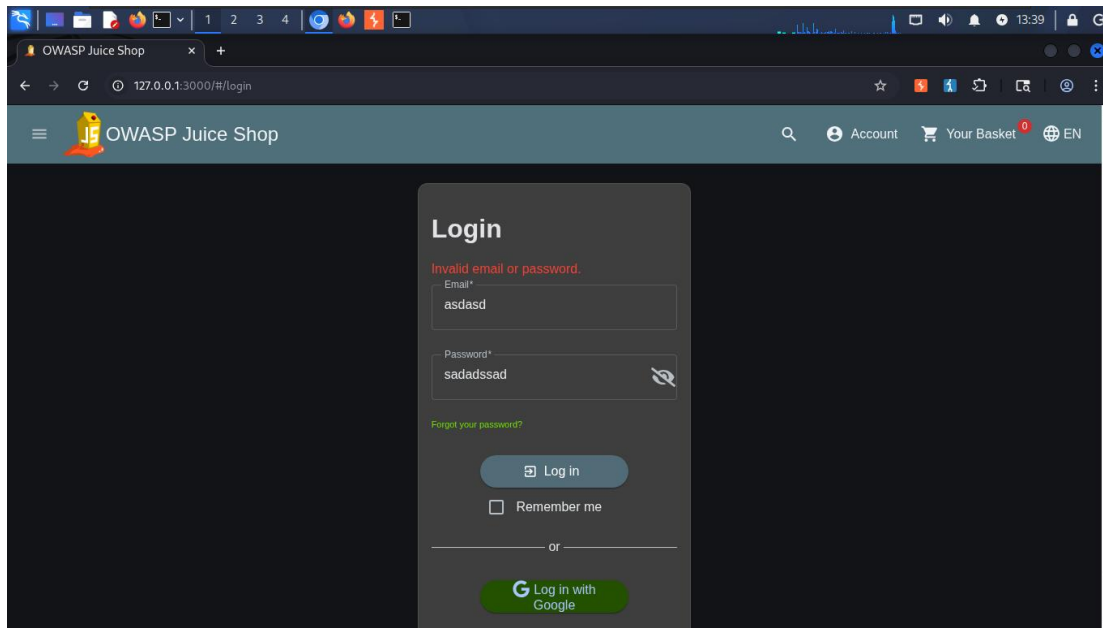
5.2 Inyección SQL (SQLi) en formulario de login

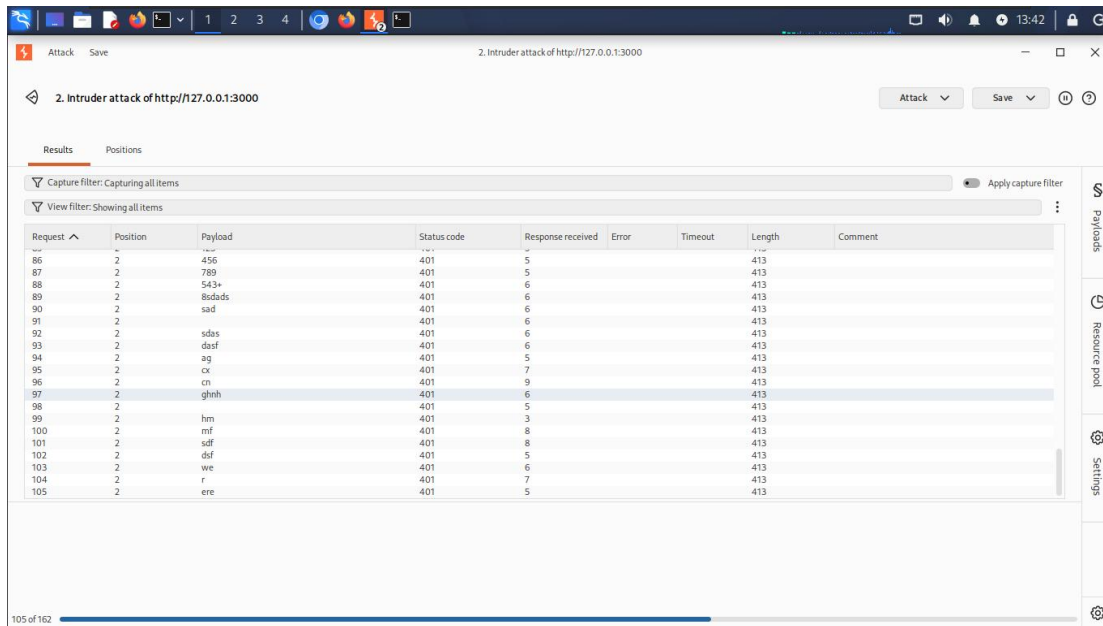
Se utilizó el payload ' OR '1'='1' -- en el campo de usuario, logrando autenticarse como administrador sin credenciales válidas.



5.3 Cross-Site Scripting (XSS)

Se inyectaron payloads de XSS reflejado y DOM-based a través del campo de búsqueda, logrando ejecución de código en el navegador de la víctima.





5.5 Resumen de hallazgos

Ataque	Resultado sin controles
SQLi	Llega a la app y puede alterar el flujo de autenticación
XSS	La entrada llega al navegador y se ejecuta sin sanitización
Fuerza bruta	Múltiples intentos permitidos sin bloqueo ni retardo
JWT	Token accesible desde el cliente / debilidad observada

6. Fase 2: Implementación de Defensa en Profundidad

A partir de los hallazgos de la Fase 1, se incorporaron capas de seguridad de forma incremental, repitiendo los mismos ataques tras cada capa para medir su efectividad.

6.1 Segmentación de red y proxy inverso

Se creó una red Docker aislada para los contenedores y se eliminó la exposición directa del puerto de la aplicación, forzando todo el tráfico a pasar por un proxy inverso Nginx.

```
Session Acciones Editar Vista Ayuda
george@kali: ~ george@kali: ~
(george@kali)-[~]
$ docker network create juice_net
31e5451d862f4ae612a73564348f1ed79a8971b89def0100e66f16d0cbe0d823
(george@kali)-[~]
$
```

```
(george@kali)-[~]
$ docker run -d \
  --name juice-shop \
  --network juice_net \
  bkimminich/juice-shop
aaa42a46039f56e965c882a55f0fa55ee1e21b1ad1ec8b2b1f4e944ed8c1ea3c
```

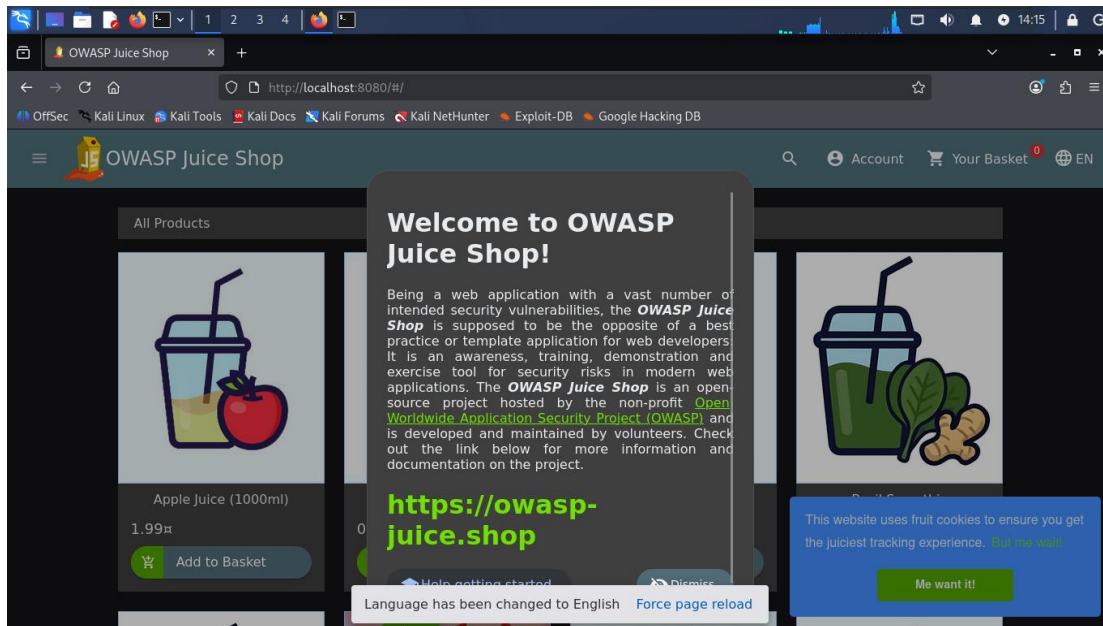
```
Session Acciones Editar Vista Ayuda
george@kali: ~ george@kali: ~
GNU nano 9.0 nginx.conf *
events {}

http {
    upstream juice_backend {
        server juice-shop:3000;
    }

    server {
        listen 80;
        server_name _;

        location / {
            proxy_pass http://juice_backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }
    }
}

Ayuda Guardar Buscar Cortar Ejecutar Ubicación Deshacer Poner marca A llave Anterior
Salir Leer fich. Reemplazar Pegar Justificar Ir a línea Rehacer Copiar Buscar atrás Siguiente
```

```
},
"ConfigOnly": false,
"Containers": {
  "56f8ba27bf990665e3cfa1448ef067318ca6911d867e58c67975ce9a1be301cd": {
    "Name": "juice-nginx",
    "EndpointID": "6c088f0fc2a3d4ce50ac837ac59605da9d9351c451e6f03f4b5f777de8a792bf",
    "MacAddress": "7a:8d:8d:33:8a:b4",
    "IPv4Address": "172.19.0.3/16",
    "IPv6Address": ""
  },
  "aaa42a46039f56e965c882a55f0fa55ee1e21b1ad1ec8b2b1f4e944ed8c1ea3c": {
    "Name": "juice-shop",
    "EndpointID": "7ca90e41b8ff1917272afad4ac77a704cfc785dae243d360b2524e43e58a32de",
    "MacAddress": "96:76:14:f2:5f:fa",
    "IPv4Address": "172.19.0.2/16",
    "IPv6Address": ""
  }
},
"Options": {},
"Labels": {}
}
```

```
(george@kali)-[~]
└─$ sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 8080/tcp
sudo ufw enable
sudo ufw status verbose
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
Rules updated
Rules updated (v6)
Firewall is active and enabled on system startup
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip
```

To	Action	From
--		
8080/tcp	ALLOW IN	Anywhere
8080/tcp (v6)	ALLOW IN	Anywhere (v6)

6.2 Limitación de tasa de peticiones en Nginx

Sobre la capa de perímetro ya establecida, se configuró control de tasa de peticiones a nivel de proxy inverso, con un límite estricto para el endpoint de autenticación y un límite general para el resto de la aplicación.

```
limit_req_zone $binary_remote_addr zone=login_limit:10m rate=5r/m;
limit_req_zone $binary_remote_addr zone=global_limit:10m rate=30r/m;

upstream juice_backend {
    server juice-shop:3000;
}

server {
    listen 80;
    server_name _;

    location /rest/user/login {
        limit_req zone=login_limit burst=3 nodelay;

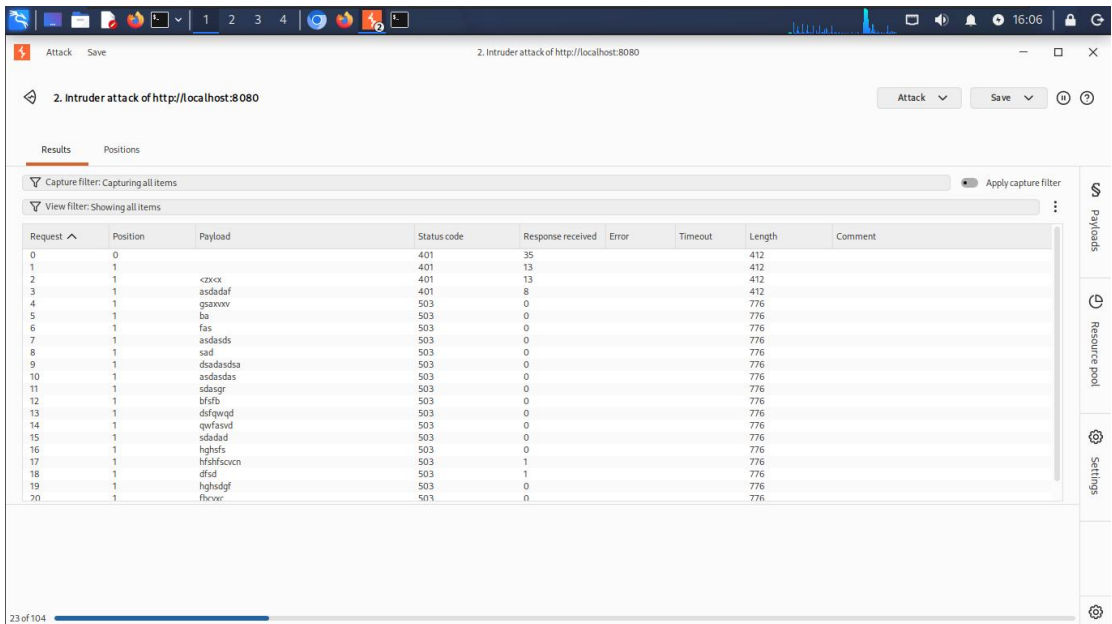
        proxy_pass http://juice_backend;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location / {
        limit_req zone=global_limit burst=20 nodelay;
```

```
(george@kali)-[~]
$ docker exec juice-nginx nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

```
(george@kali)-[~]
$ docker exec juice-nginx nginx -s reload
2026/06/24 20:56:17 [notice] 36#36: signal process started
```



6.3 Web Application Firewall (WAF)

Como capa adicional de defensa orientada a la aplicación, se desplegó un Web Application Firewall basado en ModSecurity con el OWASP Core Rule Set (CRS), posicionado igualmente como intermediario del tráfico hacia la aplicación.

```
(george@kali)-[~]
$ docker run -d \
  --name juice-waf \
  --network juice_net \
  -p 8000:80 \
  -v $(pwd)/nginx.conf:/etc/nginx/conf.d/default.conf:ro \
  owasp/modsecurity-crs:nginx
f1ceba3d2c522fbc7293028c641e86ac0b47ad643a0c719c17202e183c11d6c4
```

6.4 Resumen de hallazgos

Capa aplicada	Control implementado	Efecto observado
Perímetro / Red	Red Docker aislada, proxy inverso Nginx, firewall UFW	Se elimina el acceso directo al contenedor de la aplicación;

		todo el tráfico se centraliza en un único punto de entrada
Aplicación (tasa)	Rate limiting en Nginx	El ataque de fuerza bruta es bloqueado (HTTP 503) tras superar el umbral configurado
Aplicación (WAF)	ModSecurity y OWASP CRS	Capa adicional para inspección de payloads (SQLi/XSS) a nivel de aplicación

7. Comparativa Final: Defense in Depth - Cybersecurity Mesh

A partir de los resultados obtenidos en el laboratorio, se contrastan ambos modelos de seguridad. El laboratorio implementado corresponde íntegramente al modelo de Defensa en Profundidad, dado que se trata de un único activo (Juice Shop) protegido mediante capas concéntricas centralizadas en torno a un perímetro común (el proxy inverso). La siguiente tabla resume las diferencias conceptuales con el modelo de Cybersecurity Mesh, aplicable a escenarios distribuidos.

Criterio	Defense in Depth	Cybersecurity Mesh (CSMA)
Punto de control	Centralizado en el perímetro (proxy/firewall)	Distribuido, asociado a cada identidad o activo
Modelo de confianza	Implícita una vez dentro del perímetro	Continua y verificada (enfoque Zero Trust)
Punto único de falla	Sí: si se compromete el proxy/perímetro, el resto queda expuesto	No: el compromiso de un nodo no otorga acceso automático a los demás
Escalabilidad en entornos híbridos/cloud	Baja; requiere reforzar el perímetro por cada nuevo punto de acceso	Alta; cada nuevo activo incorpora su propio perímetro de identidad
Visibilidad	Fragmentada por capa (red, host, aplicación)	Unificada mediante dashboards y analítica consolidada
Aplicabilidad al laboratorio realizado	Total: red segmentada, proxy, firewall y rate limiting alrededor de un único activo	Conceptual: requeriría replicar el control de identidad por cada microservicio si la app se descompusiera en varios componentes distribuidos

8. Conclusiones

- La ausencia de controles permitió la explotación inmediata de SQLi, XSS y fuerza bruta sobre Juice Shop, validando la línea base de vulnerabilidad establecida en la Fase 1.
- La implementación progresiva de capas (segmentación de red, proxy inverso, firewall y rate limiting) demostró ser efectiva para contener específicamente el ataque de fuerza bruta, evidenciado por el cambio de código de respuesta de 401 a 503 tras superar el umbral configurado.
- El modelo de Defensa en Profundidad resulta adecuado para escenarios centralizados como el del laboratorio, donde existe un único activo y un perímetro claramente definible.
- El modelo de Cybersecurity Mesh se vuelve relevante cuando el escenario se distribuye en múltiples activos, identidades o entornos (cloud, remoto, microservicios), donde un perímetro único deja de ser representativo de la superficie de ataque real.
- Ambos modelos no son mutuamente excluyentes: la Defensa en Profundidad puede entenderse como la base de control por capas que, al distribuirse y centrarse en la identidad de cada activo, evoluciona hacia un enfoque de Cybersecurity Mesh en entornos más complejos.